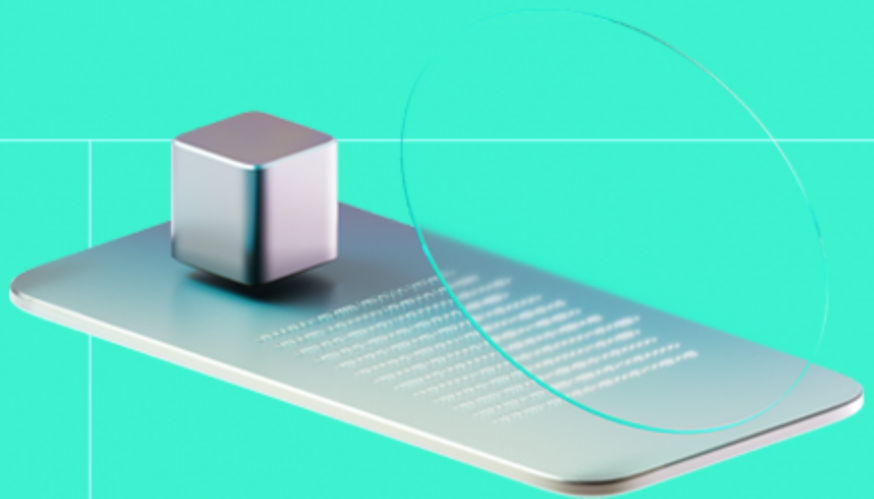




Smart Contract Code Review And Security Analysis Report

Customer: NEBA Token

Date: 17/10/2025



We express our gratitude to the NEBA Token team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

NEBA Token is a secure, upgradeable ERC-20 token implementation designed to combine regulatory readiness with strong operational controls. It integrates role-based access management, pausable transfers, and UUPS upgradeability, ensuring a controlled token environment with built-in mechanisms for emergency halts and governance-driven upgrades.

Document

Name	Smart Contract Code Review and Security Analysis Report for NEBA Token
Audited By	Panagiotis Konstantinidis, Seher Saylik
Approved By	Ataberk Yavuzer
Website	https://nebatoken.com
Changelog	10/10/2025 - Preliminary Report 17/10/2025 - Final Report
Platform	EVM
Language	Solidity
Tags	Fungible Token, ERC20
Methodology	https://hackenio.cc/sc_methodology

Review Scope

Repository	https://github.com/JayP11/NEBA-Token
Commit	27107b87fae8617d61d387b90421d90aab0efb4e
Retest	1f432d15fa540ce0b56bd64d50ef14af128ddf30

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

5	5	0	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	1
Low	2

Vulnerability	Severity	Status
F-2025-13411 - Pausing Disables Allowance Revocation Leaving Users Exposed During Emergencies	Medium	Fixed
F-2025-13407 - Initializer Could Be Front-run	Low	Fixed
F-2025-13414 - Inaccurate NatSpec on initialize() Function	Low	Fixed
F-2025-13406 - Floating Pragma	Info	Fixed
F-2025-13410 - Reentrancy Guard Included but Never Used Creates False Sense of Protection	Info	Fixed

Documentation quality

- Functional requirements are evident from the spec: fixed 1B supply at init, role-gated pausing/unpausing, and UUPS-based upgrading.
- The in-code comments provide a sufficient technical overview of access control, pause flow, and upgrade authorization.

Code quality

- Implements Solidity best practices with OpenZeppelin upgradeable libraries (ERC20, Pausable, AccessControl, UUPS), custom errors, and explicit events.
- Upgradeable deployment is correctly structured (initializer pattern, disabled constructor, storage gap, proxy-safe overrides), indicating a properly configured development approach.

Test coverage

Code coverage of the project is **100%** (branch coverage).

- All core functionalities such as deployment, pausing, unpausing, role-based access control, upgrades, and transfers are thoroughly tested.
- The development team included integration, unit, fuzz, and invariant tests, validating both normal and edge-case behaviours across multiple roles.
- Invariant tests confirm total supply consistency and prevent balance overflows, while fuzz tests simulate randomized user interactions under varying conditions.

Table of Contents

System Overview	6
Privileged Roles	6
Potential Risks	7
Findings	8
Vulnerability Details	8
Disclaimers	18
Appendix 1. Definitions	19
Severities	19
Potential Risks	19
Appendix 2. Scope	20
Appendix 3. Additional Valuables	21

System Overview

NEBAToken is an upgradeable ERC-20 compliant token designed with enhanced administrative control and safety mechanisms. It features a fixed total supply of 1 billion tokens minted at initialization and allocated to the designated treasury address. The contract implements role-based access control to separate key responsibilities, allowing specific roles to handle pausing, and upgrading while keeping minting closed post-deployment.

It has the following attributes:

- **Name:** NEBA Token
- **Symbol:** NEBA
- **Decimals:** 18
- **Total Supply:** 1,000,000,000 tokens

Privileged roles

- The `DEFAULT_ADMIN_ROLE` of the `NEBAToken` contract holds full administrative privileges, including the ability to grant and revoke any other role within the system. This role effectively governs the token's access control and overall management.
- The `ADMIN_PAUSER_ROLE` can pause and unpause the contract, enabling or restoring token transfers during emergency or maintenance situations.
- The `BOT_PAUSER_ROLE` is limited to pausing the contract and is intended for automated systems or bots to halt token activity in response to predefined triggers or abnormal behavior.
- The `UPGRADER_ROLE` is authorized to upgrade the contract's implementation through the UUPS proxy mechanism, granting the ability to deploy new logic versions while retaining the existing contract state.

Potential Risks

- **Centralized Role Control:** `DEFAULT_ADMIN_ROLE`, `ADMIN_PAUSER_ROLE`, and `UPGRADER_ROLE` concentrate authority in core functionalities. Compromise or misuse can pause transfers, reassign roles, or change logic.
- **Flexibility and Risk in Contract Upgrades:** The UUPS pattern allows logic upgrades at any time by the `UPGRADER_ROLE`, creating trust and governance risk if the upgrader is compromised or deploys faulty code.
- **Absence of Upgrade Window Constraints:** No on-chain timelock or mandatory delay precedes upgrades, increasing the chance of rapid deployment of unvetted implementations.
- **Insufficient On-Chain Multi-Signature Controls:** The contract does not enforce multi-signature for privileged actions. The team states a multi-signature wallet will be used operationally, which should materially reduce this risk once implemented.
- **Fixed Supply Central Allocation:** The entire 1,000,000,000 NEBA supply is minted to the admin treasury at initialization, creating a single point of control over initial distribution.
- **Out-of-Scope Components:** Off-chain deployment scripts, governance processes, and any future upgrade implementations are not covered by this audit and may introduce additional risk if misconfigured or unaudited.

Findings

Vulnerability Details

F-2025-13411 - Pausing Disables Allowance Revocation Leaving Users Exposed During Emergencies - Medium

Description:

The token intentionally pauses transfers via `ERC20PausableUpgradeable`. In addition, approvals are also gated by the `whenNotPaused` modifier through a custom `_approve()` override. This deviates from OpenZeppelin's default `ERC20Pausable` behavior, which does not pause approvals.

```
// Approvals are disabled while paused
function _approve(address owner, address spender, uint256 value, bool emitEvent)
internal
virtual
override
whenNotPaused // <-- blocks *all* allowance changes during pause
{
    super._approve(owner, spender, value, emitEvent);
}
```

During a pause, users often want to rapidly reduce or zero-out allowances granted to DEXs, routers, custodians, bots, or compromised third-party keys. Because `_approve()` function is guarded with the `whenNotPaused` modifier, allowance changes—including reductions to `0`—are rejected while the token is paused.

As a result, users remain exposed to stale or compromised spenders for the duration of a pause, exactly when risk is elevated (e.g., a DEX/router exploit discovered, API key leak, or phishing event).

Assets:

- contracts/NEBAToken.sol [<https://github.com/JayP11/NEBA-Token>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: Adopt a revocation-friendly pause. Keep transfers paused, but allow non-increasing allowance changes (i.e., reductions or setting to 0) while paused. This can be enforced in the `_approve()` override by permitting decreases when `paused() == true`, and only blocking increases.

Example Fix:

```
function _approve(address owner, address spender, uint256 value, bool emitEvent)
internal
virtual
override
{
    if (paused()) {
        // Allow revocations / reductions during pause; block increases
        uint256 current = allowance(owner, spender);
        require(value <= current, "Approvals can only decrease while paused")
    }
    super._approve(owner, spender, value, emitEvent);
}
```

Alternative remove the pause check from approvals entirely and gate only transfers with pause by deleting `whenNotPaused` modifier from `_approve()` function and keeping it solely on `_update()` function.

Resolution: The `approve()` function does not enforce an unpaused status now within the new implementation. Revised commit: `1f432d1`.

[F-2025-13407](#) - Initializer Could Be Front-run - Low

Description:

In Solidity smart contracts, initializers are functions used to set up initial state variables or perform essential setup tasks during contract deployment. However, a vulnerability arises when initializers are not adequately protected. If an initializer can be front-run, meaning an attacker can execute it before legitimate deployment, several security risks emerge.

The contract defines a public `initialize()` function meant to be called once during deployment. This function uses the `initializer` modifier to ensure it can only be called once. Although the initializer modifier enforces single execution, any new instance that is created without invoking `initialize()` in the same transaction is exposed to front-running by external actors. Since no access control is enforced on this function beyond the initializer check, any user monitoring mempool transactions can potentially invoke `initialize()` first and take control of the contract's state.

Within the contract, the `initialize()` function is public and is missing proper access control, which gives a chance for front-run initialization to occur:

```
function initialize(address adminTreasury, address upgraderAddress, address adminPauser, address botAddress)
    public
    initializer
    {
        ...
        _mint(adminTreasury, INITIAL_SUPPLY);
    }
```

This pattern is a common risk in upgradeable proxy-based deployments. Until the `initialize()` function is executed, the contract remains uninitialized, exposing a window during which ownership and control of the entire system can be hijacked. If an attacker successfully front-runs this call, they gain full administrative privileges, including the ability to upgrade the contract and manipulate its intended functionality.

Even if the team's deployment process atomically calls `initialize()` via the `ERC1967Proxy` constructor, removing any front-running window, this is a procedural guarantee rather than a code-level invariant. The on-chain contract does not itself enforce atomic initialization for future instances. Therefore, any future instance created without

being initialized in the same transaction could be taken over by the first caller.

Assets:

- contracts/NEBAToken.sol [<https://github.com/JayP11/NEBA-Token>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Medium

Severity: Low

Recommendations**Remediation:**

To mitigate the risk of front-running the `initialize()` function, restrict access to the initializer so that only a trusted entity, typically the deployer, can invoke it. This can be achieved by adding a check within the function that ensures the caller matches a predefined address.

Alternatively, ensure that the initialization is performed atomically during or immediately after proxy deployment, such as within the same transaction via a deployment script or factory contract. These measures prevent unauthorized users from hijacking the initialization process and gaining privileged control over the contract.

Resolution:

The NEBA Token team stated that the initialization process is already executed during the deployment phase.

[F-2025-13414](#) - Inaccurate NatSpec on initialize() Function - Low

Description:

The `initialize()` function contains a NatSpec block that does not match the function's actual interface and logic. Specifically:

- The current comments omit the `@param adminPauser` description even though the function takes the `address adminPauser` parameter and grants it the `ADMIN_PAUSER_ROLE`. This omission can mislead integrators and users about how to supply parameters, increasing the chance of incorrect initialization.
- The comment claims "ADMIN_PAUSER_ROLE and BOT_PAUSER_ROLE have the ability to pause and unpause the token", but the implementation permits only `ADMIN_PAUSER_ROLE` to unpause. The `BOT_PAUSER_ROLE` can only pause.
- The comments state "BOT_PAUSER_ROLE has the ability to pause the token for bots," which implies a scoped bot-only effect, while the code's `pause()` affects all transfers globally.

```
/**
 * @notice Initializes the NEBA Token contract and grants roles to the adminT
reasury
 * DEFAULT_ADMIN_ROLE has all permissions and it can grant and revoke roles
 * ADMIN_PAUSER_ROLE and BOT_PAUSER_ROLE have the ability to pause and unpau
e the token
 * BOT_PAUSER_ROLE has the ability to pause the token for bots
 * UPGRADER_ROLE has the ability to upgrade the contract
 * @param adminTreasury Address for day-to-day management (admin, pauser).
 * @param upgraderAddress Address that can upgrade the contract (should be a
separate, high-security multisig/wallet).
 * @param botAddress Address for the automated keeper bot (can only pause).
 * @dev Mints entire supply to adminTreasury and sets up roles
 */
function initialize(
    address adminTreasury,
    address upgraderAddress,
    address adminPauser, // ← present in code, missing in NatSpec
    address botAddress
) public initializer { ... }

function pause() external onlyPausers whenNotPaused { _pause(); }
function unpause() external onlyRole(ADMIN_PAUSER_ROLE) whenPaused { _unpause
(); }
```

These documentation mismatches create misleading expectations around operational controls and emergency procedures.

Assets:

- contracts/NEBAToken.sol [<https://github.com/JayP11/NEBA-Token>]

Status:**Fixed**

Classification**Impact Rate:** 3/5**Likelihood Rate:** 2/5**Exploitability:** Semi-Dependent**Complexity:** Simple**Severity:** **Low**

Recommendations**Remediation:**

Update the NatSpec to accurately reflect parameters and role semantics, and keep it synchronized with code behavior:

- Add `@param adminPauser` explaining it receives `ADMIN_PAUSER_ROLE` (unpauses; can also pause).
- Correct the role description: Both `ADMIN_PAUSER_ROLE` and `BOT_PAUSER_ROLE` can pause, but only `ADMIN_PAUSER_ROLE` can unpause.
- Clarify that `BOT_PAUSER_ROLE` triggers a global pause (not “for bots only”).

Example Fix:

```
/**
 * @notice Initializes NEBA and configures roles & supply.
 * @param adminTreasury Receives INITIAL_SUPPLY and DEFAULT_ADMIN_ROLE.
 * @param upgraderAddress Address granted UPGRADER_ROLE.
 * @param adminPauser Address granted ADMIN_PAUSER_ROLE (can pause and unpause).
 * @param botAddress Address granted BOT_PAUSER_ROLE (can pause only).
 * @dev Pausing is global; both pauser roles can pause; only ADMIN_PAUSER_ROLE can unpause.
 */
```

Resolution:

The NatSpec is corrected by the NEBA Token team. Revised commit: `1f432d1`.

[F-2025-13406](#) - Floating Pragma - Info

Description:

In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.20`) introduces uncertainty, as it allows the contract to be compiled with any version within the specified range. This can result in discrepancies between the compiler used in testing and the one used during deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The project currently uses floating pragma declarations (`^0.8.20`) in its Solidity contracts. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

```
pragma solidity ^0.8.20;
```

Assets:

- contracts/NEBAToken.sol [<https://github.com/JayP11/NEBA-Token>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

It is recommended to lock the pragma version to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler

version, preventing unexpected changes in behavior due to compiler updates. For example, instead of using `^0.8.x`, explicitly define the version with a locked, stable pragma directive.

Before selecting a version, review known bugs and vulnerabilities associated with each Solidity compiler release. This can be done by referencing the official Solidity compiler release notes: [Solidity GitHub releases](#) or [Solidity Bugs by Version](#). Choose a compiler version with a good track record for stability and security.

Resolution:

The pragma is locked to “**pragma solidity 0.8.30**”. Revised commit: `1f432d1` .

F-2025-13410 - Reentrancy Guard Included but Never Used

Creates False Sense of Protection - Info

Description:

The `NEBAToken` contract inherits OpenZeppelin's `ReentrancyGuardUpgradeable` and initializes it, but no external state-changing function is marked with the `nonReentrant` modifier. As written, the reentrancy guard provides no runtime protection, since it doesn't block reentrant entry into any code path, but its presence suggests the contract is protected:

```
import "@openzeppelin/contracts-upgradeable/utils/ReentrancyGuardUpgradeable.  
sol";  
  
...  
  
contract NEBAToken is  
    ERC20Upgradeable,  
    ERC20PausableUpgradeable,  
    AccessControlUpgradeable,  
    UUPSUpgradeable,  
    ReentrancyGuardUpgradeable  
{  
    ...  
}
```

This is risky for an upgradeable token since future upgrades might add external calls (e.g., hooks, bridges, or callbacks) under the assumption that a guard is in place, leaving those paths reentrancy-exposed. It also bloats bytecode and complicates the code with unused security overhead..

Assets:

- `contracts/NEBAToken.sol` [<https://github.com/JayP11/NEBA-Token>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	2/5
Exploitability:	Dependent
Complexity:	Simple

Severity:

Info

Recommendations**Remediation:**

If reentrancy protection is intended, apply `nonReentrant` to any externally callable, state-changing function that could (now or in future upgrades) perform external calls or token transfers. Otherwise remove the `ReentrancyGuardUpgradeable` inheritance and initializer to avoid dead code and confusion. Additionally, document the reentrancy posture to guide future upgrades.

Resolution:

The `nonReentrant` modifier is now used for pause/unpause functions. Revised commit: `1f432d1`.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/JayP11/NEBA-Token
Commit	27107b87fae8617d61d387b90421d90aab0efb4e
Retest	1f432d15fa540ce0b56bd64d50ef14af128ddf30
Whitepaper	N/A
Requirements	docs, README.md
Technical Requirements	docs, README.md, NEBAToken.md

Asset	Type
contracts/NEBAToken.sol [https://github.com/JayP11/NEBA-Token]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Additional Considerations

This security assessment was conducted in accordance with the [NIST SP 800-115 - Technical Guide to Information Security Testing and Assessment](#). This guideline provides a structured foundation for planning, executing, and documenting technical evaluations such as testing, vulnerability assessments, and code reviews. Hacken's internal methodology and security assessment framework extend these principles to blockchain environments to ensure consistency, repeatability, and verifiable outcomes.